Predictive Modeling for Bank Marketing Campaigns

May 13, 2025

Abstract

In this project, we develop and evaluate predictive models to estimate the likelihood that a client of a banking institution will subscribe to a term deposit, based on the Bank Marketing dataset. Our workflow encompasses comprehensive data preprocessing—including handling of sentinel values, correction of distributional skew, standardization of numeric attributes, and one-hot encoding of categorical variables. We compare seven classifiers—logistic regression, K-nearest neighbors, decision tree, random forest, naive Bayes, support vector machine, and a feed-forward neural network (64–32 ReLU layers with sigmoid output)—using stratified train/test splits and class weighting to address the positive-class imbalance. Performance is assessed in terms of accuracy, F1-score, precision, and recall.

These findings demonstrate that careful feature engineering—especially sentinel handling, skew reduction, and imbalance mitigation—significantly improves predictions in financial marketing contexts.

Contents

1 Introduction **3**

2 Dataset Description **3**
2.1 Dataset Breakdown 3
2.2 Dataset Imbalance 3

3 Exploratory Data Analysis (EDA) **4**
3.1 Descriptive Analysis 4
3.2 Correlation Analysis 5

4 Dataset Pre-processing **6**
4.1 Handling Null Values 6
4.2 Handling Categorical Values 6
4.3 Label Encoding and One-Hot Encoding 6
4.4 Handling Skewness 6
4.5 Dataset Splitting 6

5 Model Training and Testing **7**

6 Model Comparison **7**
6.1 Performance Metrics 7
6.2 Confusion Matrices 8
6.3 ROC and AUC 8
6.4 Neural Network Loss Curve 9

7 Conclusion **10**
7.1 Model-wise Conclusions 10
7.2 Overall Conclusion 10
7.3 Reasons behind Conclusion 10
7.4 Challenges Faced 10
7.5 Final Verdict 11

1 Introduction

This project focuses on building a machine learning model that predicts whether a customer will subscribe to a specific programme based on their personal and campaign-related information. The model can efficiently identify potential customers who are likely to respond positively. The motivation behind this project is to enable data-driven decision-making and optimize the cost of marketing campaigns by targeting likely responders.

2 Dataset Description

2.1 Dataset Breakdown

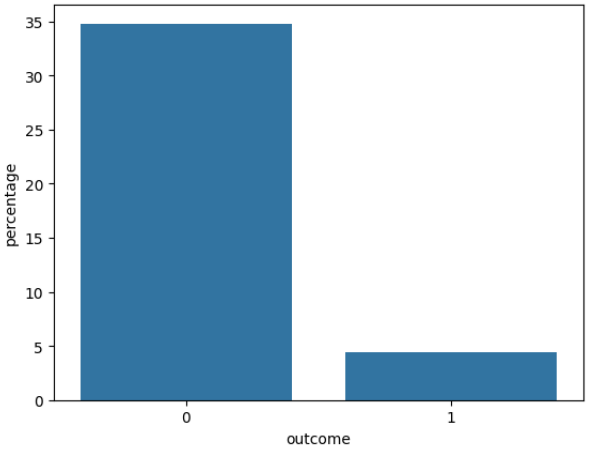
The dataset comprises 41,188 instances and 20 features (10 categorical, 10 quantitative), with a binary target variable `y` indicating subscription ("yes"/"no").

Table 1: Categorical and Numerical Features in the Dataset

Categorical Features	**Numerical Features**
job	age
marital	duration
education	campaign
default	pdays
housing	previous
loan	emp.var.rate
contact	cons.price.idx
month	cons.conf.idx
day_of_week	euribor3m
poutcome	nr.employed

2.2 Dataset Imbalance

The output feature is imbalanced in the dataset as the unique classes do not have an equal number of instances. In the dataset the class 'no' appears significantly more than the class 'yes'.:



(a) Outcome Target Imbalance

Outcome	Count	Percentage
No (0)	36 548	88.73%
Yes (1)	4 640	11.27%

(b) Class distribution of the target variable

Figure 1: Target imbalance: (a) visual depiction and (b) numeric breakdown

3 Exploratory Data Analysis (EDA)

3.1 Descriptive Analysis

We computed summary statistics and visualized distributions with histograms and boxplots for numeric features.

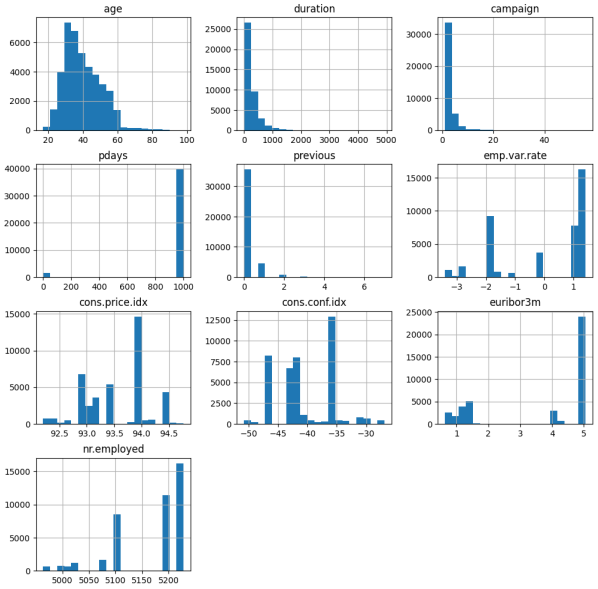


Figure 2: Features Histogram

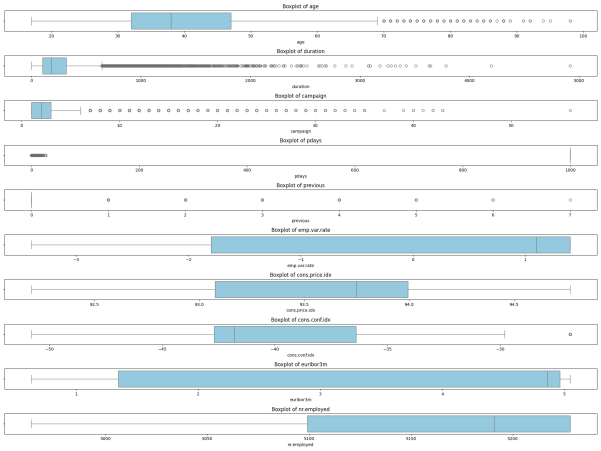


Figure 3: Box and Whisker Plot

3.2 Correlation Analysis

From the correlation test, values closer to 0 (typically between -0.3 and 0.3) indicate little to no linear correlation, while values closer to ± 1 (especially above 0.9 or below -0.9) indicate strong positive or negative correlation. From the heatmap we see there are weak or no correlation between some features and also there are highly correlated values.

Table 2: Correlation between Features

Feature 1	Feature 2	Correlation Coefficient
emp.var.rate	euribor3m	0.972
emp.var.rate	nr.employed	0.907
euribor3m	nr.employed	0.945

There are highly correlated values and carry similar information which make the models redundant and affect the performance of the models. Correlation test helps in selecting relevant features and removing redundant ones to improve model efficiency.

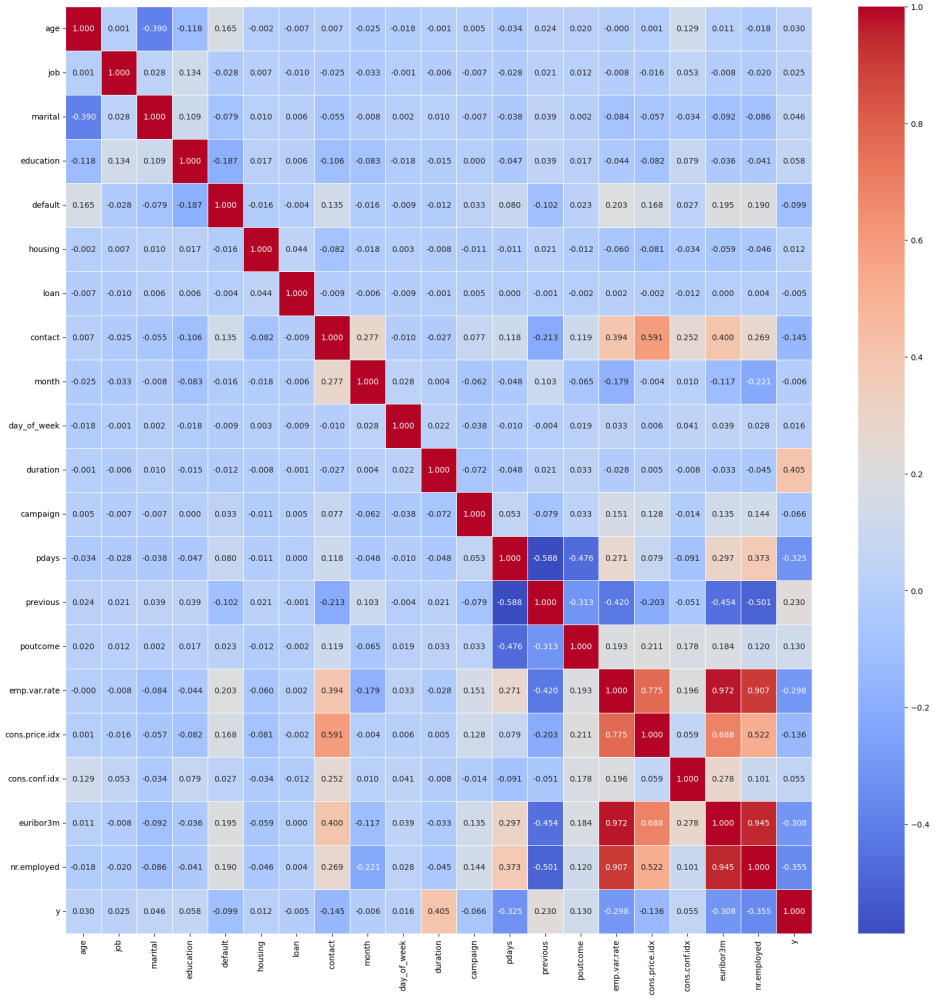


Figure 4: Correlation Matrix Heatmap

4 Dataset Pre-processing

4.1 Handling Null Values

No nulls were present initially. However, after handling `pdays`, null values were generated in `pdays_notcontacted` columns, which were then imputed with median values.

4.2 Handling Categorical Values

The `pdays` feature was transformed: a binary variable `pdays_contacted` marks whether a client was contacted (`pdays` $\neq$ 999), and a log-transformed variable `pdays_notcontacted` captures non-contacted clients (with 999 treated as missing). The original `pdays` was dropped.

4.3 Label Encoding and One-Hot Encoding

The target `y` was labelencoded to 0/1. All remaining categorical features were one-hot encoded with `drop_first=TRUE` to avoid multicollinearity.

4.4 Handling Skewness

All numeric columns (`int64` and `float64`) were inspected for skewness. Those with a skewness greater than 1 were flagged as highly skewed. For each skewed column (excluding the target `y`) that contained only non-negative values, a log-plus-one transformation (`log1p`) was applied to reduce skew and stabilize variance.

4.5 Dataset Splitting

- Due to high class imbalance, data were stratified during the train–test split (70% train / 30% test) to ensure balanced class distributions:

Table 3: Train–Test Split Sizes

Dataset	**Rows**
Train Data	12 357
Test Data	28 831

- Applied SMOTE (Synthetic Minority Over-sampling Technique) to balance the dataset so the model learns equally from both classes.
- Further balanced the learning by passing `class_weight='balanced'` to each model.

5 Model Training and Testing

Seven classification algorithms were trained and evaluated. Models were assessed on accuracy, precision, recall, and F1-score.

Table 4: Overview of Classification Models and Configurations

Algorithm	**Configuration**
Logistic Regression	`max_iter`=5000, `class_weight`='balanced'
K-Nearest Neighbors (KNN)	`n_neighbors`=4
Decision Tree	`class_weight`='balanced'
Random Forest	`class_weight`='balanced', `random_state`=42
Naive Bayes	GaussianNB (default parameters)
Support Vector Machine (SVM)	`probability`=True, `class_weight`='balanced'
Feed-Forward Neural Network (NN)	2 hidden layers: 64, 32 units (ReLU); output sigmoid; optimizer=Adam(lr=0.001); loss=binary_crossentropy; metrics=[accuracy]; epochs=40; batch_size=20

Linear Regression was not applied because it is a classification problem, where the target variable is categorical ("yes"/"no"), and linear regression is designed for predicting continuous numerical outcomes.

6 Model Comparison

6.1 Performance Metrics

Bar charts compare accuracy, precision, recall, and F1-scores across models. Precision and recall scores determine how well each model identifies positive cases. Logistic Regression and Random Forest performed well, with Logistic Regression showing the best recall, making it suitable for detecting actual positives.

Table 5: Comparison of Models on Key Performance Metrics (Class = Yes)

Model	**Precision**	**Recall**	**F1–Score**	**Accuracy**
Logistic Regression	0.41	**0.87**	0.55	0.84
K-Nearest Neighbors (KNN)	0.36	0.55	0.44	0.84
Decision Tree	0.46	0.57	0.51	0.88
Random Forest	**0.58**	0.62	**0.60**	**0.91**
Naive Bayes	0.24	0.77	0.37	0.71
Support Vector Machine	0.46	0.81	0.58	0.87
Neural Network	0.49	0.59	0.54	0.87

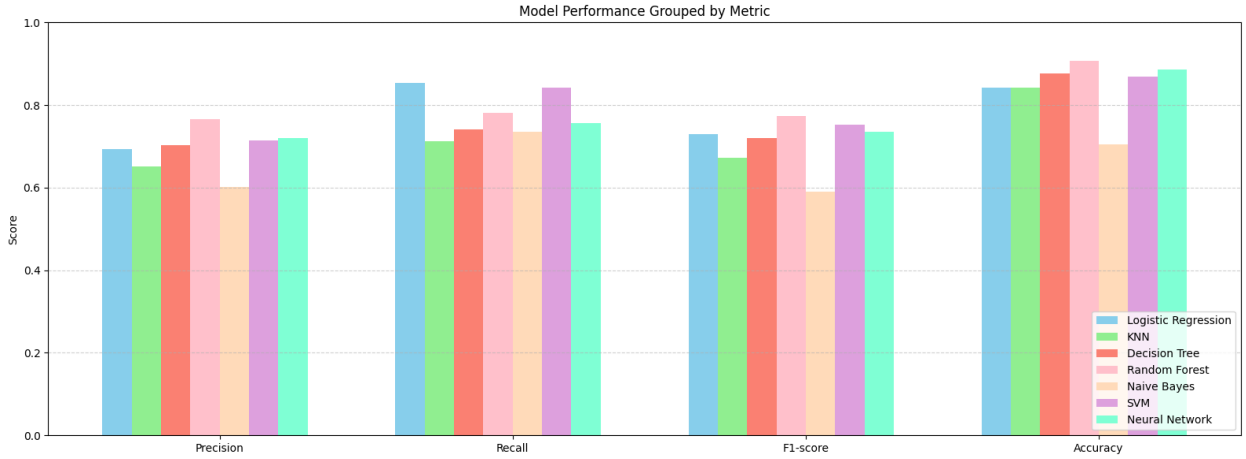


Figure 5: Model Evaluation

6.2 Confusion Matrices

Confusion matrices quantified true/false positives and negatives for each model.

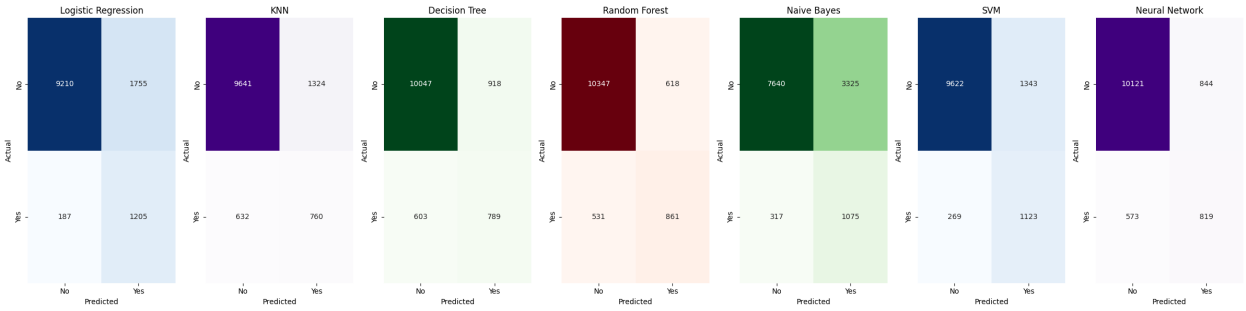


Figure 6: Confusion Matrices

6.3 ROC and AUC

ROC curves and AUC scores provided an overview of model performance.

Table 6: Model Performance (AUC Scores)

Model	**AUC**
Logistic Regression	0.92
K-Nearest Neighbors (KNN)	0.78
Decision Tree	0.74
Random Forest	0.94
Naive Bayes	0.81
Support Vector Machine	0.93
Neural Network	0.91

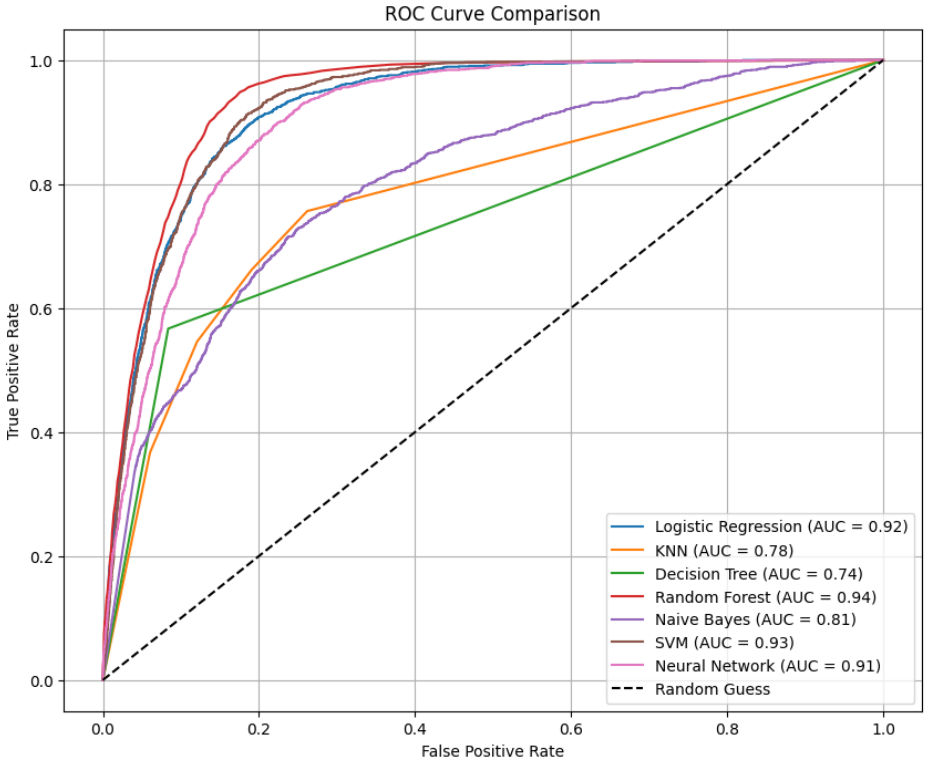


Figure 7: ROC and AUC graph

6.4 Neural Network Loss Curve

To monitor the training dynamics of our feed-forward neural network, we capture the training loss over each epoch and plot it. This visualization helps to verify that the model is converging and to detect issues such as under- or over-fitting.

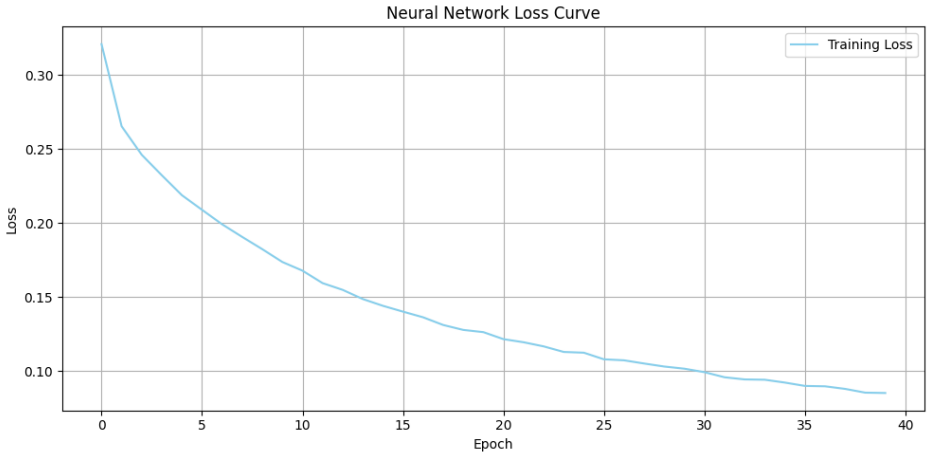


Figure 8: Neural Network Loss Curve

7 Conclusion

7.1 Model-wise Conclusions

- Random Forest: Best overall performance (high accuracy and AUC).
- Logistic Regression: Highest recall, interpretable.
- KNN: Lower precision and recall due to class imbalance.
- Decision Tree: Moderate performance.
- Naive Bayes: Poor performance (feature dependencies).
- SVM: Balanced precision and recall.
- Neural Network: Good but slightly behind ensemble methods.

7.2 Overall Conclusion

Random Forest performed strongly in terms of accuracy, precision, and AUC score, indicating its effectiveness in correctly classifying both classes. However, Logistic Regression achieved the highest recall, making it more suitable for this classification problem where correctly identifying the positive class (e.g., customers likely to subscribe) is more important.

7.3 Reasons behind Conclusion

We got such results because models like random forest and SVM handle imbalanced data better than other models. Logistic Regression handles linear relationships effectively which increases the recall. Naive Bayes performs poorly as there are feature dependencies. KNN also struggles due to the high imbalance, which affects its distance based classification.

7.4 Challenges Faced

One of the major challenges faced during the project was dealing with highly imbalanced data, which required careful analysis to ensure that models did not become biased toward the majority class. Another key challenge was selecting relevant features to avoid overfitting and improve generalization. Additionally, handling the `pdays` feature was tricky, as many values were set to 999, which skewed the results if not addressed separately. .

7.5 Final Verdict

Table 7: Performance Summary by Metric

Aspect	**Best Model**	**Remarks**
Accuracy	Random Forest	Highest overall accuracy (0.91), strong on majority-class performance
Precision (Yes)	Random Forest	Best precision (0.60) for predicting 'Yes', fewer false positives
Recall (Yes)	Logistic Regression	Best at identifying actual 'Yes' cases (Recall = 0.87)
F1-Score (Yes)	Random Forest	Best trade-off between precision and recall (F1 = 0.60)
Overall Best	*See below*	Choice depends on whether recall or overall balance (F1) is more critical

Overall best model depends on priority:

- **For Maximizing Recall:** Use **Logistic Regression** (Recall = 0.87) to catch the most positives.
- **For Maximizing Balance (F1):** Use **Random Forest** (F1 = 0.60) for the best harmonic mean of precision and recall.